Dynamic Provider Discovery — Design Spec

Date: 2026-06-11 **Status:** Approved (operator decisions recorded 2026-06-11) **Owner:** Lava client + lava-api-go + Tracker SDK

1. Problem

The Lava Android client hardcodes its provider/tracker list: `TrackerClientModule.provideTrackerRegistry()` (`core/tracker/client/.../di/TrackerClientModule.kt:263-281`) manually registers **7** providers, and `TrackerDescriptor`s are compiled-in Kotlin singletons. The chosen API instance has no say in what providers the client offers, and Jackett's indexers (an arbitrary, deployment-specific set) are invisible to the client.

The API already executes **all** provider work server-side — `lava-api-go` routes every operation through `/v1/{provider}/{search,browse,topic,download,login,…}` gated by capability middleware (`internal/handlers/v1/handlers.go:60-104`) backed by a `ProviderRegistry` (`internal/provider/registry.go`) — but exposes **no discovery endpoint**, so the client cannot ask the API "what providers do you support, and how do I authenticate to each?"

2. Goal

The client populates its provider list **and** per-provider auth/sign-in UI **dynamically** from the chosen API instance. Every provider the API supports — including **every configured Jackett indexer** — becomes a first-class, fully-usable provider in the client, working with all client features the provider's declared capabilities allow. No false results; everything covered by tests + Challenges + HelixQA with captured evidence.

3. Operator decisions (2026-06-11)

1. **Client architecture: Thin API-backed client.** One generic `ApiBackedTrackerClient` handles ANY provider id by calling the API. Every discovered provider — incl. all Jackett indexers —

works through it with zero new per-provider Kotlin parsers. The compiled-in registry becomes an offline fallback.
2. **Jackett model: each indexer = a provider.** The API enumerates Jackett's configured indexers and exposes each as its own discoverable provider.
3. **Submodule sync: deliberate + build-verified.** Pull code-dependency submodules to latest, rebuild + test after each wave; the `constitution` submodule advances via its own CONST-049 ceremony.

4. Architecture

4.1 API side (lava-api-go, embedded in api-app)

New endpoint `GET /v1/providers` → returns the full provider catalogue:

```
{
  "providers": [
    {
      "id": "rutracker",
      "displayName": "RuTracker.org",
      "kind": "native",
      "capabilities":
        ["SEARCH","BROWSE","FORUM","TOPIC","COMMENTS","FAVORITES","TORRENT_DOWNLOA
      "authType": "CAPTCHA_LOGIN",
      "encoding": "Windows-1251",
      "baseUrls": ["https://rutracker.org","https://
        rutracker.net"],
      "supportsAnonymous": false
    },
    {
      "id": "1337x",
      "displayName": "1337x",
      "kind": "jackett",
      "indexer": "1337x",
      "capabilities": ["SEARCH","MAGNET_LINK","TORRENT_DOWNLOAD"],
      "authType": "NONE",
      "encoding": "UTF-8",
      "baseUrls": [],
      "supportsAnonymous": true
    }
  ]
}
```

- Built from `registry.All()`. Each provider's metadata comes from the `Provider` interface (`ID/DisplayName/Capabilities/AuthType/Encoding`) plus a new `Kind()/SupportsAnonymous()/BaseURLs()` surface (added to the interface with safe defaults for existing providers).

- The handler lives at `internal/handlers/v1/providers.go`; registered in `handlers.go` BEFORE the `/:provider/` group so `/v1/providers` is not shadowed by the provider middleware. Capability-free (no provider middleware). §6.AC telemetry on the error path.
- Added to `api/openapi.yaml` (spec-first; regenerate `internal/gen/server`).

Jackett indexers as registry providers:

- New `internal/jackett/indexers.go`: `ListIndexers(ctx) ([]IndexerInfo, error)` calling Jackett's `GET /api/v2.0/indexers?configured=true&apikey=…` (apikey server-side only, §6.H). `IndexerInfo = {ID, Name, Caps}`.
- New `internal/provider/jackettprovider/provider.go`: `JackettIndexerProvider` implements `provider.Provider`. `ID()=<indexer-id>`, `Kind()="jackett"`, `AuthType()=NONE`, `Capabilities()={SEARCH, MAGNET_LINK, TORRENT_DOWNLOAD}`. `Search()` delegates to the existing Jackett client with its indexer id; `DownloadFile()` delegates to `jackett.Client.Download`. Extended caps (browse/topic/comments/favorites/login) return the standard 501 path (capability honesty, §6.E).
- At startup (`internal/router/router.go` wiring, when `JackettEnabled`): enumerate indexers, register one `JackettIndexerProvider` per indexer into the registry. **Collision guard:** if an indexer id equals a native provider id, the native provider wins and a §6.AC warning is recorded (the jackett one is skipped).
- The existing `/jackett/search` handler is retained for back-compat but is no longer the primary path; Jackett providers now flow through the uniform `/v1/{id}/{op}` routing.

4.2 Client side (Android)

Wire DTO + remote descriptor (`core/tracker/api` or `core/network/dto`):

- `@Serializable data class ProviderDescriptorDto(id, displayName, kind, indexer?, capabilities: List<String>, authType: String, encoding, baseUrls: List<String>, supportsAnonymous: Boolean)`.
- `RemoteTrackerDescriptor(dto): TrackerDescriptor` — maps the DTO into the existing `TrackerDescriptor` interface (capabilities→`Set<TrackerCapability>`, authType→`AuthType`, baseUrls→`List<MirrorUrl>`, `apiSupported=true`, `verified=true` since the API vouches for it).

Catalogue repository (`core/data`):

- `ProviderCatalogRepository.fetchProviders(apiBaseUrl): List<RemoteTrackerDescriptor>` → `GET /v1/providers` via the existing network layer; parses, maps, returns. Persists the catalogue (Room or preferences) keyed by API instance so the

app renders the list on next cold start without a round-trip; refresh on-demand + on API switch.
- `FetchProvidersUseCase` in `core:domain`.

Generic API-backed client (`core/tracker/client`):

- `ApiBackedTrackerClient(descriptor, networkApi): TrackerClient` implementing the feature interfaces (`SearchableTracker`, `BrowsableTracker`, `TopicTracker`, `DownloadableTracker`, `AuthenticatableTracker`, …) by calling `/v1/{descriptor.trackerId}/{op}` through the network layer. `getFeature<T>()` returns a non-null impl iff the descriptor declares the matching capability (capability honesty, §6.E mirror).
- `DynamicTrackerRegistry` (or `DefaultTrackerRegistry.populateFrom(descriptors)`): given the fetched descriptors, register one `ApiBackedTrackerClient` each. Replaces the static 7-factory registration as the runtime source of truth; the compiled-in 7 remain a bundled fallback used when the catalogue fetch fails or for the default/offline API.

Onboarding wiring (`feature/onboarding`, `feature/login`):

- After the ApiSelection step's connectivity probe succeeds, call `FetchProvidersUseCase(apiBaseUrl)` → populate the registry → advance to provider selection. `ProviderLoginViewModel.loadProviders()` already reads `sdk.listAvailableTrackers()`; with the registry now dynamically populated it shows the API's list.
- Auth UI (`ProviderCredentialForm`) is already generic by `AuthType` — verify NONE/API_KEY/FORM_LOGIN/ CAPTCHA all render from a dynamic descriptor; add API_KEY field rendering if missing.

4.3 Data flow

```
Onboarding: choose API → probe /health → GET /v1/providers
   → [ProviderDescriptorDto] → [RemoteTrackerDescriptor] →
registry.populateFrom(...)
   → provider list UI (dynamic) → user selects provider P
   → auth UI rendered from P.authType → login (if needed) via /v1/
P/login
   → search/browse/topic/download via /v1/P/{op}
(ApiBackedTrackerClient)
Jackett indexer P=<indexer>: identical path; server delegates to
the Jackett sidecar.
```

5. Error handling

- Discovery fetch failure → fall back to the bundled 7 descriptors + a non-blocking "couldn't reach API, showing bundled providers" notice; never a blank screen (the §6.AB rendering-correctness lesson).
- A declared-but-unimplemented capability → API returns 501; client surfaces "not supported by this provider" rather than a crash (capability honesty both sides).
- Jackett enumeration failure at API startup → API still serves native providers; Jackett providers simply absent from the catalogue (logged via §6.AC), not a hard failure.
- All new error paths on both sides record non-fatal telemetry (§6.AC).

6. Testing strategy (hard evidence, no bluff — §6.J/§6.L/§6.Z)

Go (lava-api-go):

- unit: discovery handler shape; registry incl. jackett providers; `ListIndexers` parsing; collision guard.
- contract (`tests/contract`): real HTTP `GET /v1/providers` returns the catalogue; each listed provider's declared capabilities resolve (no 501 for a declared cap) — extends the §6.E parity gate.
- e2e (`tests/e2e`): discover → search → download for a native provider AND a Jackett indexer, real stack.
- parity (`tests/parity`): catalogue shape stable.
- Each new test carries the Bluff-Audit falsifiability stamp.

Kotlin (unit, JVM):

- `ProviderDescriptorDto` parse; `RemoteTrackerDescriptor` mapping; `DynamicTrackerRegistry.populateFrom`; `ApiBackedTrackerClient` routes the right URL + `getFeature` capability gating (real MockWebServer, not mocked SUT); onboarding ViewModel fetches + renders the dynamic list + falls back on fetch failure.

Compose UI Challenge tests (device, Genymotion via Containers/§6.AH):

- New `Challenge39DynamicProviderDiscoveryTest`: onboarding → choose API → provider list is populated FROM the API (assert a provider that only the API knows about appears) → select it → search → real result row.
- New `Challenge40JackettIndexerProviderTest`: a Jackett indexer appears as a provider → search → download.

- Falsifiability rehearsal per Challenge (break the discovery fetch → list empty/fallback → test fails).

HelixQA QA sessions:

- A vision-guided QA session (`scripts/run-helixqa-challenges.sh`) driving the dynamic onboarding + provider use end-to-end, evidence under `.lava-ci-evidence/.../helixqa-challenges/`.

Evidence: every gate run captures verbatim output (BUILD SUCCESSFUL / ok / canary verdicts) under `.lava-ci-evidence/`. No metadata-only claims.

7. Submodule sync (deliberate, build-verified)

Per wave: `git -C submodules/<x> pull --ff-only`, rebuild the affected surface, run the affected tests; on green, bump the pin + commit. Order: leaf code deps (auth, cache, concurrency, config, database, ratelimiter, recovery, security, http3, mdns, middleware, observability) → containers → challenges/helixqa → tracker_sdk. The `constitution` + `panoptic` submodules advance via their own ceremonies (CONST-049), handled separately and may introduce new mandatory rules to adopt.

8. Out of scope (YAGNI)

- Per-Jackett-indexer category filtering UI (the API can pass categories later; not needed for "usable").
- Per-indexer auth (Jackett is app→sidecar→indexer; the rare per-indexer-auth case is deferred).
- OAuth provider flows (no current provider uses OAUTH; the enum value stays but no UI is built now).

9. Versioning / distribute

Feature ships under the already-bumped next-cycle versions (client 1.3.3-1060, api-app 0.2.3-7, api-go 2.3.25-2325) with auth rotation + §6.AA two-stage + §6.Z device verification at distribute time, per the established release runbook. Distribute is a separate operator-gated step after the feature lands.

Post-1076 Revision (2026-06-26)

1076 Distribute Outcome

Lava-Android-1.3.12-1076 + api-app 0.2.11-22 were distributed and reported to testers. The operator's post-distribute manual testing concluded "practically almost nothing has been fixed". Root cause: the §6.Z device gate executed **only Challenge00CrashSurvivalTest (C00)** while the cycle's CHANGELOG claimed fixes to search, provider-selection, onboarding, and display. The repo contains 60+ Challenge tests (C01–C57) covering those exact flows, and **none were executed** before distribute. The cold-start canary proved nothing about the shipped user-visible value. This is now a constitutionally prohibited pattern per **§6.AK** (Cycle-Coverage Device Gate, added 2026-06-26): a `Challenge00CrashSurvivalTest`-only gate NEVER satisfies the distribute requirement when the cycle claims flow/UI fixes.

Current State of `/v1/providers`

The `GET /v1/providers` endpoint is live and working on the on-device api-app. The real catalogue exposes **12 providers** — 5 natives (rutracker, rutor, nnmclub, kinozal, archiveorg) + 7 curated (gutenberg, thepiratebay, yts, torrentscsv, knaben, nyaa, bitsearch, torrentdownloads — note gutenberg shipped before this spec was written). This exceeds the 7-jackett-estimate the original architecture section assumed; the dynamic-discovery mechanism is proven by the actual provider count. The 1077-dev cycle will reconcile the client's hardcoded "4 providers" welcome text to match reality.

Video-Issue Feedback (LVA-079–LVA-091)

Ten open video issues from the operator's manual-testing session identify concrete problems that affect the spec's assumptions:

- **Search broken** — multiple root causes identified: auth-interceptor overwriting the handoff key (H1, fixed in 1072 as commit `627a0d58`), engine 18s deadline vs client 30s read-timeout mismatch producing silent failures, partial-failure Error→Empty display path (fixed in `1cbf364c`). A dynamic provider that cannot complete search is not usable by the user regardless of catalogue correctness.
- **Provider filter chips disagree** between search-input and results screens — the source-of-truth for which providers are active is not consistent. This affects the dynamic-discovery data flow (section 4.3) because `ApiBackedTrackerClient` routes through whichever provider ID the chip layer sends.

- **Display labels raw/lowercased** — a provider's `displayName` from the DTO must be title-cased properly; the current rendering path on the client does not normalize backend labels.
- **API selection screen mislabeled** — the on-device API discovery flow (ApiSelection step) uses a misleading label that confuses users.
- **mDNS discovered API shows raw IP** — the network-discovered API surface presents the raw IPv4 address rather than a human-readable name.

§6.AK Impact on Testing Strategy

Section 6's testing strategy must be updated to reflect the new §6.AK gate requirements. Every claimed fix for any issue affecting the dynamic discovery flow now demands an **EXECUTED + PASSED** covering device Challenge before the version can be distributed:

- `Challenge39DynamicProviderDiscoveryTest` (described above) is mandatory before any distribute that claims dynamic-discovery fixes.
- The falsifiability rehearsal for this Challenge must demonstrate a RED run against the pre-fix build (e.g. discovery fetch returns empty → test fails) before the GREEN run against the fix.
- The `Challenge00CrashSurvivalTest` alone is NEVER sufficient (§6.AK clause 1). The §6.Z evidence file for any dynamic-discovery cycle must enumerate the executed Challenge FQNs matching the CHANGELOG claim-set.
- The operator's video-issue backlog (LVA-079–LVA-091) must be addressed as a per-video reproduction set per §6.AK clause 5 before the next distribute that claims "search fixed" or "provider discovery fixed."

Known Issues Affecting Spec Assumptions

- **LVA-008:** The `C11` ConnectedAndroidTest navigation-teardown crash blocks nested-route device Challenges. Until this is resolved, any device Challenge that navigates through multiple screens (like the dynamic-discovery full flow: onboarding → API selection → provider selection → search → results) risks producing a false-negative due to test-infrastructure flakiness rather than a production bug.
- **Search engine timeout mismatch:** The engine's configured 18s deadline and the client's 30s read-timeout produce a window where the engine times out internally but the client keeps waiting. Dynamic providers whose search response takes >18s will fail even though the provider is correctly registered.

- **AuthInterceptor handoff-key overwrite (H1):** Fixed in 1072, but the same class of middleware-state corruption could affect the dynamic provider's per-request authentication headers.
- **Partial-failure Error→Empty display fix:** Landed in `1cbf364c`. Before this fix, a partial failure (some providers returning results, others erroring) rendered as an empty screen rather than showing the successful results alongside the error. This affects the user-visible quality of the dynamic-discovery search flow.

Updated Change History

Date	Author	Change
2026-06-11	Agent	Initial spec
2026-06-26	Agent	Post-1076 revision: documented C00-only gate outcome, updated provider count to 12, added video-issue feedback, §6.AK testing gate, known issues, and this change history entry